Gradient Descent

Mathematics, Types, Algorithms, and Visualizations

A detailed guide to gradient descent optimization, including mathematical foundations, different variants (Batch, Stochastic, Mini-batch), common pitfalls, learning rates, and supporting visuals.

Autor: Maru (assisted by Grok)

February 2026

Gradient Descent Overview

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. It is widely used in machine learning to minimize loss functions.

The algorithm iteratively updates parameters in the direction opposite to the gradient of the function, scaled by a learning rate α.

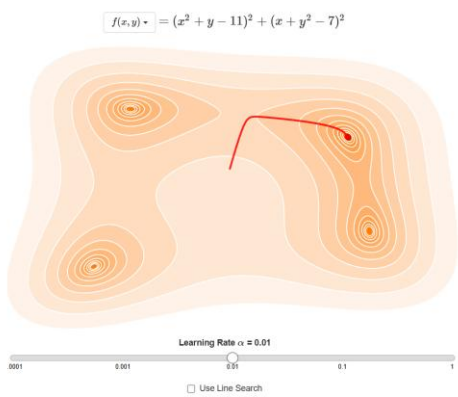

Figure 1: Basic Gradient Descent trajectory on a convex cost function.

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta J(\theta^{(t)})$$

Equation 1: Parameter update rule in gradient descent.

Mathematical Foundations

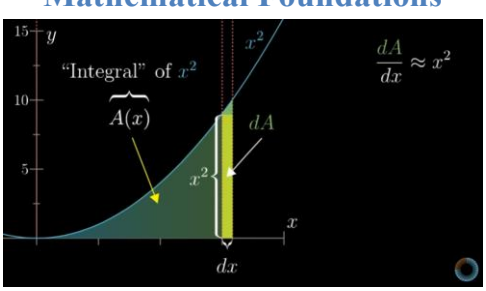

Figure 2: Visualization of the derivative as slope.

$$f'(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$$

Equation 2: Definition of the derivative.

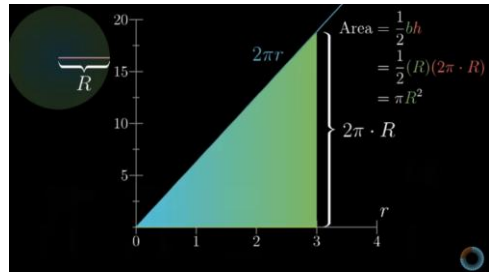

Figure 3: Geometric interpretation of derivative via area approximation.

The core update rule moves parameters towards the minimum by subtracting the gradient times the learning rate.

This process repeats until convergence criteria are met, such as small gradient norm or plateau in loss.

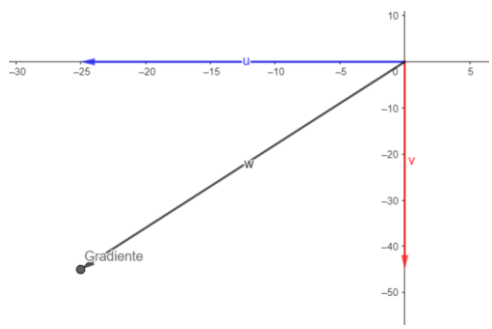

Figure 4: Gradient vector in multivariable function.

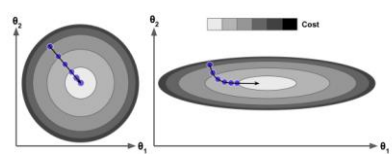

Figure 5: Gradient descent with two features (2D).

Types of Gradient Descent

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} J(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\theta) \end{bmatrix}$$

Equation 3: Gradient vector for multivariable function.

The derivative measures how much the function changes with a small change in input. For optimization, the slope indicates the steepest descent direction.

Negative slope means increase input to decrease function value.

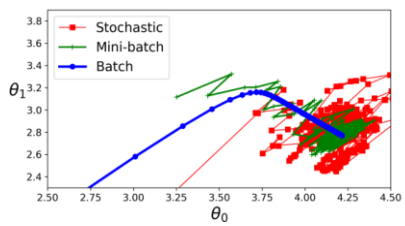

Figure 6: Comparison of Batch (blue), Mini-batch (green), Stochastic (red) GD paths.

Common Pitfalls and Solutions

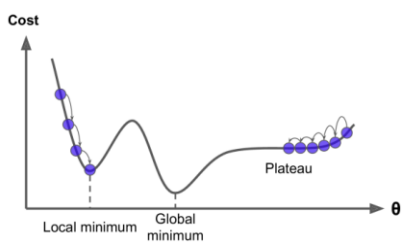

Figure 7: Common pitfalls: local min, plateau, saddle points.

In multivariable calculus, the gradient is a vector of partial derivatives pointing to the steepest ascent.

Gradient descent follows the negative gradient for descent.

Learning Rate Selection

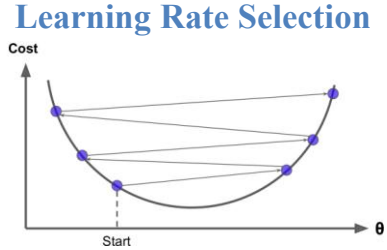

Figure 8: Effect of large learning rate causing divergence.

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Equation 4: Example mean squared error loss (linear regression).

Batch Gradient Descent computes the gradient using the entire dataset. It provides a stable but computationally expensive update.

Ideal for small datasets.

- Provides accurate gradient estimate
- Smooth convergence path
- Guaranteed decrease if α small enough
- Computationally expensive (O(N))
- Requires full dataset in memory
- Slow for large datasets

Stochastic Gradient Descent (SGD) uses one training example per update. It is noisy but faster and can escape local minima.

Common in deep learning.

- Fast updates
- Low memory usage
- Can escape local minima via noise
- Noisy updates
- Zigzag path
- Variance in convergence

Mini-batch GD strikes a balance, using small batches (e.g., 32-256 samples). It leverages vectorized operations in frameworks like TensorFlow.

- Balances speed and stability
- GPU-friendly batching
- Default in most frameworks

Table 1: Comparison of Gradient Descent Variants

Type	Batch Size	Pros	Cons	Use Cases
Batch GD	All (N)	Smooth, accurate	Slow, memory-heavy	Small datasets
Stochastic GD	1	Fast, escapes locals	Noisy	Online learning
Mini-batch GD	32-1024	Balanced, efficient	Tunable batch size	Deep learning

- Local minima traps
- Plateaus (vanishing gradients)
- Saddle points
- Exploding/vanishing gradients

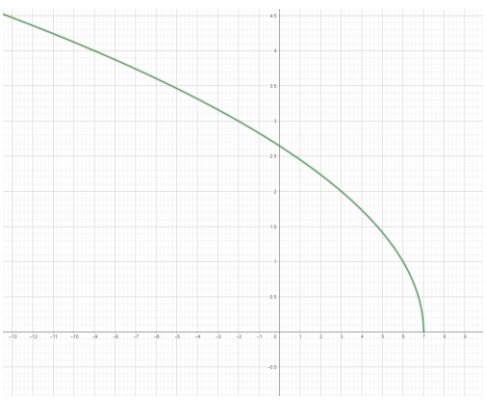

Figure 9: Function domain in 1D and 3D surfaces.

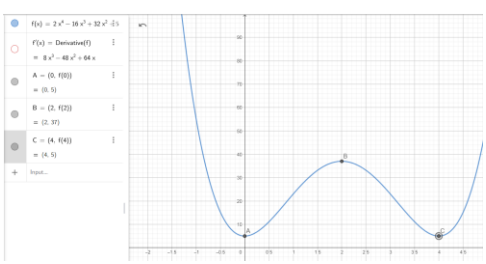

Figure 10: First derivative test for extrema.

Advanced Topics and Conclusion

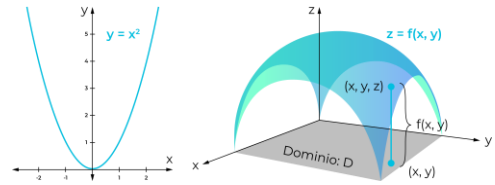

Figure 11: Contour plot showing GD path and local minima.

The learning rate α controls step size. Too large causes divergence; too small leads to slow convergence.

Adaptive methods like Adam adjust α dynamically.

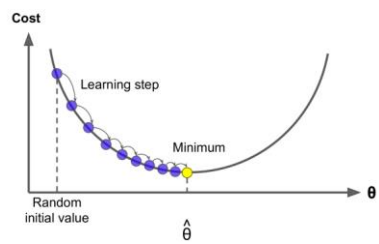

Figure 12: Gradient computation in 3D visualization.

- Momentum
- Adaptive optimizers (RMSprop, Adam)
- Learning rate scheduling
- Batch normalization

$$\theta_{t+1} = \theta_t - \frac{\eta}{||g_t|| + \epsilon} g_t(RMSprop)$$

Equation 5: Adaptive learning rate in RMSprop.

In conclusion, gradient descent and its variants are foundational to modern ML. Understanding their math and pitfalls ensures effective implementation.

Experiment with hyperparameters for best results.